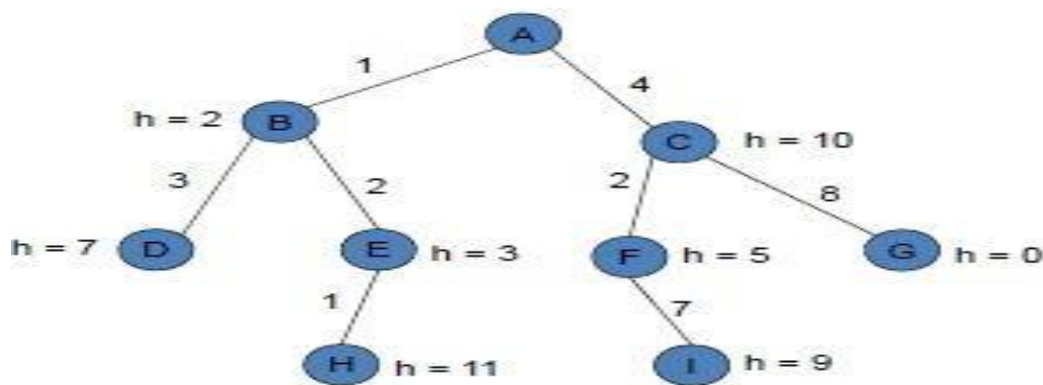


Corrigé TP-4, Algorithmes de recherche informée

Exercice 1 :

Dans quel ordre les nœuds sont développés pour chacun des algorithmes ?

- Meilleur d'abord glouton
- A*



Solution :

Meilleur d'abord glouton : {A, B, E, D, C, G}

A* :

$$f(B) = g(B) + h(B) = 1 + 2 = 3$$

$$f(C) = g(C) + h(C) = 4 + 10 = 14$$

$$f(D) = g(D) + h(D) = 3 + 1 + 7 = 11$$

$$f(E) = g(E) + h(E) = 2 + 1 + 3 = 6$$

$$f(F) = g(F) + h(F) = 4 + 2 + 5 = 11$$

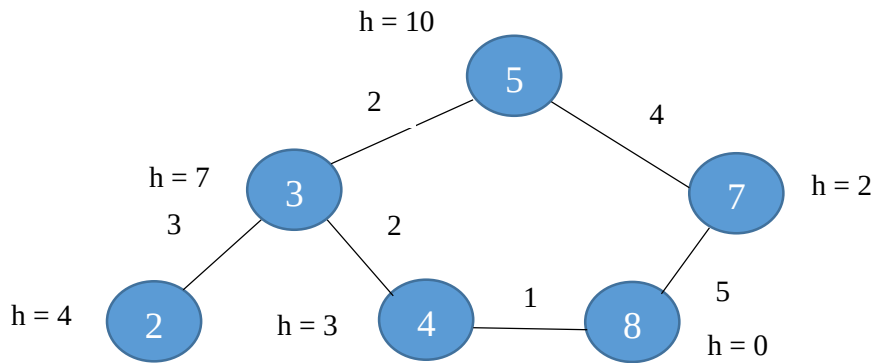
$$f(H) = g(H) + h(H) = 1 + 2 + 1 + 11 = 15$$

$$f(I) = g(I) + h(I) = 4 + 2 + 7 + 9 = 22$$

La séquence : {A, B, E, D, C, G}

Exercice 2 :

En utilisant les algorithmes, Meilleur d'abord glouton et A*, chercher le nœud 8.



Meilleur d'abord glouton :

```

nombre_noeuds = 6
graphe = [[] for i in range(nombre_noeuds)]
def fct(x,y,cout):
    graphe[x].append((y,cout))
    graphe[y].append((x, cout))
fct(0,1,7)
fct(0,2,2)
fct(1,3,4)
fct(1,4,3)
fct(2,5,0)
fct(4,5,0)
print(graphe)
from queue import PriorityQueue
liste = [5,3,7,2,4,8]
def best_first(graphe,node,goal,n):
    visited = [False]*n
    visited[node] = True
    p_queue = PriorityQueue()
    p_queue.put((0,node))
    while p_queue.empty() == False:
        m = p_queue.get()[1]
        print(liste[m])
        if m == goal:
            return 1
        for n,c in graphe[m]:
            if visited[n] == False:
                visited[n] = True
                p_queue.put((c,n))
    return 0
print("%d"%best_first(graphe,0,5,6))

```

A* : Il faut modifier les couts avec les nouveaux couts calculer avec la fonction $f(n) = g(n) + h(n)$

$$f(3) = g(3) + h(3) = 7 + 2 = 9$$

$$f(7) = g(7) + h(7) = 4 + 2 = 6$$

$$f(2) = g(2) + h(2) = 3 + 4 + 2 = 9$$

$$f(4) = g(4) + h(4) = 2 + 2 + 3 = 7$$

Donc les instructions à modifier sont :

fct(0,1,9)

fct(0,2,6)

fct(1,3,9)

fct(1,4,7)

fct(2,5,0)

fct(4,5,0)